Principal Component Analysis (PCA): Identify the variables in your dataset that are suitable for Principal Component Analysis (PCA).

- Suitable: the six-monthly bill amounts (BILL_AMT1 to BILL_AMT6).
- They are numerical and highly correlated (0.80 to 0.96), with VIF values above 10, so they carry mostly the same information. An unpaid balance carries into the next month's bill, so the six columns measure one thing: how much the customer owes. PCA can combine them into fewer components.

Not suitable:

- PAY_AMT1 to PAY_AMT6, LIMIT_BAL and AGE: numerical, but not strongly correlated with each other, so there is little to combine.
- PAY_0, PAY_2 to PAY_6: ordinal repayment-status codes; the gaps between the values are not real distances.
- SEX, EDUCATION and MARRIAGE: categorical variables.
- ID and the target: ID is only a row number, and the target must never be used as an input (data leakage).

Provide sample Python code demonstrating how PCA can be applied to the selected variables.

```
from sklearn.decomposition import PCA

bill_columns = ["BILL_AMT1", "BILL_AMT2", "BILL_AMT3","BILL_AMT4", "BILL_AMT5", "BILL_AMT6"]

pca = PCA()

X_train_pca = pca.fit_transform(X_train_scaled) # fit on train only

X_test_pca = pca.transform(X_test_scaled)

explained_variance = pca.explained_variance_ratio

cumulative_variance = np.cumsum(explained_variance)

print(cumulative_variance)


loadings = pd.DataFrame(pca.components _. T, index=bill_columns)

print(loadings)
```

The number of components kept will be the smallest number that explains at least 90% of the variance,

checked with a scree plot. The loadings show which bill columns contribute to each component.

Data Standardisation: Explain how you plan to standardise the selected variables before applying PCA.

- The data will be split into a training set (80%) and a test set (20%) before anything is learned from it.
- LIMIT_BAL and PAY_AMT1 to PAY_AMT6 are very skewed, so np.log1p will be applied to them first. The bill amounts will not be log transformed because they contain negative values (credit balances).
- StandardScaler will be used to give every variable a mean of 0 and a standard deviation of 1: z = (x -mean) / standard deviation.
- This is needed because PCA is sensitive to scale. Without it, variables with large values (bill amounts in the hundreds of thousands) would dominate variables with small values like AGE.
- The scaler will be fitted on the training set only and then applied to the test set, so no test information leaks into the preparation.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train[bill_columns])
X_test_scaled = scaler.transform(X_test[bill_columns])
```

Feature Selection: Identify the feature selection techniques you plan to apply to your dataset.

- Near-constant check: find columns where one value appears in 95% or more of the rows.
- Correlation with the target: corrwith(y_train) to see which features are related to default.
- Dummy variable trap: drop one EDUCATION and one MARRIAGE dummy column.
- Backward elimination using VIF: remove the feature with the highest VIF, recalculate, and repeat until every VIF is below 5.
- PCA vs original columns: compare keeping some of the bill columns with replacing them by the first principal component.
- Gini importance: a small decision tree, used only to rank the features.

Explain why these methods are appropriate

- **Near-constant check**: a column with almost one value carries little information. Rare columns will be checked rather than removed automatically.
- **Correlation with the target**: shows which features are useful for predicting default (expected: the most recent repayment status, PAY_0).
- **Dummy variable trap**: the one-hot columns always add up to 1, so one of them can always be worked out from the others.
- **Backward elimination using VIF**: the bill amounts repeat each other. Removing one feature at a time matters because VIF changes after each removal; removing all of them at once would lose the bill information completely.
- **PCA vs original columns**: the brief asks to compare the two. An original column such as “most recent bill” is easier to explain than a component.
- **Gini importance**: a simple way to rank features for a classification problem. It is not used to evaluate a model, and its limits are that it favours features with many different values and splits importance between correlated features.
- Together these cover the three reasons to remove a feature: it carries no information, it repeats another feature, or it is not related to default. None of them need a final model, and all of them use the training set only.